Project Report

CSP 1602: Software Engineering Lab

Spendly - AI-Powered Expense Tracker & Habit Coach

Submitted by

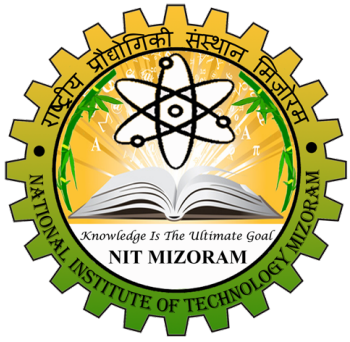
Shivam Dwivedi

Roll No.: BT23CS010

Under the guidance of

Dr. C Lalengmawia

Assistant Professor, Department of CSE



National Institute of Technology Mizoram

Department of Computer Science & Engineering

Aizawl – 796012, Mizoram, India

April 2026

Abstract

Traditional personal finance applications often act as passive repositories for transaction data, lacking the proactive intelligence required to cultivate lasting financial discipline. This project presents Spendly, a cross-platform (iOS and Android) AI-powered expense tracker and habit coach designed to bridge this gap. Built using React Native and Expo, Spendly integrates a robust Node.js/Express backend powered by a Neon Serverless PostgreSQL database. The core innovation lies in the Average Spending Analysis (ASA) engine and its integration with the Groq Large Language Model (llama-3.1-8b-instant). By injecting real-time analytics, habit streaks, and behaviour scores into system prompts, the AI financial coach provides highly contextualised, actionable insights to the user. Features include intuitive transaction logging, per-category budgeting with 80% threshold alerts, automated recurring transactions, and native PDF report generation. The resulting system not only tracks expenses but actively coaches users toward better financial habits through predictive alerts and a discipline scoring metric.

Keywords: Personal Finance, AI Coach, Expense Tracker, React Native, Large Language Models, Average Spending Analysis.

Acknowledgement

I would like to express my sincere gratitude to my project guide, Dr. C Lalengmawia, Assistant Professor, Department of Computer Science & Engineering, NIT Mizoram, for his invaluable guidance, continuous support, and constructive feedback throughout the development of this project.

I also extend my thanks to the Department of Computer Science & Engineering, NIT Mizoram, for providing the necessary resources and environment to successfully complete this software engineering lab project.

Shivam Dwivedi
Roll No.: BT23CS010
April 2026

Contents

Abstract i

Acknowledgement ii

1 Introduction 1
- 1.1 Background . . . 1
- 1.2 Purpose of the System . . . 1
- 1.3 Definitions and Acronyms . . . 1
- 1.4 Product Functions . . . 1
- 1.5 Problem Statement . . . 2
- 1.6 Objectives . . . 2
- 1.7 Scope and Limitations . . . 2
- 1.8 Organisation of the Report . . . 3

2 Related Work 4
- 2.1 Existing Systems and Approaches . . . 4
- 2.2 Tools and Technologies Reviewed . . . 4
- 2.3 Gap Identified . . . 4

3 Methodology 5
- 3.1 System Architecture . . . 5
  - 3.1.1 Architecture Explanation . . . 5
- 3.2 System Workflow . . . 6
- 3.3 Detailed Functional Requirements . . . 6
  - 3.3.1 Authentication and Profile . . . 6
  - 3.3.2 Transaction Management . . . 6
  - 3.3.3 Budget Management . . . 6
  - 3.3.4 AI Financial Coach . . . 7
  - 3.3.5 ASA Analytics Engine . . . 7
  - 3.3.6 Additional Features . . . 7
- 3.4 External Interface Requirements . . . 7
  - 3.4.1 User Interface . . . 7
  - 3.4.2 Software Interfaces . . . 7
  - 3.4.3 Communication Interface . . . 7
- 3.5 API Specification . . . 8
- 3.6 Database Design . . . 8
- 3.7 System Design Models . . . 9
  - 3.7.1 Use Case Description . . . 9
- 3.8 Data Flow Overview . . . 10
  - 3.8.1 Data Flow Explanation . . . 10
- 3.9 Design Decisions . . . 10
- 3.10 Non-Functional Requirements . . . 11
  - 3.10.1 Performance . . . 11
  - 3.10.2 Security . . . 11
  - 3.10.3 Scalability . . . 11

3.10.4 Usability 11

4 Results and Discussion **12**
4.1 Experimental Setup / Environment 12
4.2 Results 12
4.3 Discussion 12

5 Conclusion **13**
5.1 Summary 13
5.2 Future Work 13

A Technical Implementation Details **15**

Chapter 1

Introduction

1.1 Background

Effective personal financial management is critical for economic stability and achieving long-term goals. While numerous expense-tracking applications exist, most require heavy manual input and function merely as digital ledgers. The emergence of Large Language Models (LLMs) presents an opportunity to evolve these tools from passive trackers into proactive financial coaches. Spendly leverages these technologies to analyze user behaviour, track discipline, and provide real-time, context-aware financial advice.

1.2 Purpose of the System

The purpose of Spendly is to provide an intelligent personal finance management system that goes beyond traditional expense tracking. It enables individuals to not only record transactions but also receive real-time AI-driven financial coaching based on their behaviour patterns and spending habits. The document and system are designed for developers, QA engineers, designers, and academic evaluators who require a structured and scalable solution for financial habit improvement.

1.3 Definitions and Acronyms

- **SRS:** Software Requirements Specification.
- **ASA Engine:** Average Spending Analysis; a proprietary backend module computing discipline scores, predictions, and alerts.
- **LLM:** Large Language Model, specifically the Groq API (llama-3.1-8b-instant).
- **JWT:** JSON Web Token used for stateless authentication (HS256).
- **ORM:** Object-Relational Mapper; Drizzle ORM is utilized for type-safe database queries.
- **Neon:** Serverless PostgreSQL cloud database provider.

1.4 Product Functions

The system provides the following major functionalities:

- **Authentication:** Register/login with bcrypt passwords and JWT sessions.
- **Transaction Management:** Full CRUD for income, expenses, and transfers with detailed metadata (payment mode, debt tracking, intent).

- **Budget Management:** Per-category monthly limits with an 80% threshold notification flag.
- **AI Financial Coach:** Injects live ASA analytics into Groq prompts to parse JSON actions.
- **ASA Analytics Engine:** Computes discipline scores, impulse scores, predictions, and proactive alerts.
- **Recurring Transactions:** Rule-based auto-transactions (daily/weekly/monthly/yearly).
- **Savings Goals & Habits:** Goal progress allocation and daily logging streak updates.
- **Reporting:** On-device PDF generation via expo-print and native sharing.

1.5 Problem Statement

Existing personal finance tracking systems lack intelligent, proactive coaching capabilities. Users are often left to interpret raw financial charts without actionable guidance, leading to poor habit retention and impulse spending. This project develops Spendly, an intelligent, cross-platform expense habit coach that not only tracks daily transactions but computes discipline scores and offers real-time, LLM-powered financial coaching to curb impulse buying and manage budgets effectively.

1.6 Objectives

- **Objective 1:** Develop a cross-platform mobile application using React Native to facilitate seamless daily tracking of income, expenses, and savings goals.
- **Objective 2:** Design and implement an Average Spending Analysis (ASA) engine to compute user discipline scores, predict month-end projections, and generate financial alerts.
- **Objective 3:** Integrate an AI Financial Coach using the Groq API that utilizes live ASA analytics to provide personalised, conversational financial guidance.
- **Objective 4:** Implement automated features such as recurring transaction scheduling and native PDF financial report generation.

1.7 Scope and Limitations

Spendly is designed for iOS and Android platforms, backed by a REST API and a serverless PostgreSQL database. It encompasses manual transaction entry, dynamic budgeting, habit streak tracking, and AI-driven insights. **Limitations:** All AI coaching is exclusively dependent on the external Groq API, and database queries rely on Neon Serverless PostgreSQL, thus requiring an active internet connection.

1.8 Organisation of the Report

Chapter 2 reviews related work and technologies. Chapter 3 details the methodology, architecture, detailed functional requirements, and API specifications. Chapter 4 presents the results and discusses the system's performance. Chapter 5 concludes the report and outlines future directions for the software.

Chapter 2

Related Work

2.1 Existing Systems and Approaches

Several mobile applications dominate the personal finance sector, including Mint, YNAB (You Need A Budget), and PocketGuard. YNAB utilizes a zero-based budgeting methodology, which is highly effective but imposes a steep learning curve on new users. Mint offers automated bank syncing and pie-chart analytics but lacks conversational, personalized coaching [**?**]. Recent advancements in generative AI have prompted research into integrating LLMs into financial advice systems, yet most existing implementations operate as standalone chatbots without deep, programmatic access to the user's live financial data [**?**].

2.2 Tools and Technologies Reviewed

To construct a highly responsive and cross-platform architecture, **React Native (Expo)** was selected over native frameworks to ensure iOS/Android compatibility. For the backend, **Node.js with Express** ensures non-blocking handling of REST APIs. **Neon Serverless PostgreSQL** persists all data, interfaced via **Drizzle ORM** for type-safe query building. The **Groq API (llama-3.1-8b-instant)** was selected as the AI engine to guarantee structured JSON output required for action routing.

2.3 Gap Identified

There is a clear gap between static expense ledgers and standalone AI chatbots. Spendly bridges this gap by injecting a proprietary pre-computed analytics snapshot (the ASA engine) directly into the LLM's system prompt on every chat interaction. This allows the AI to trigger native application actions (e.g., `ADD_TX`, `SET_BUDGET`) rather than just generating plain text.

Chapter 3

Methodology

3.1 System Architecture

Spendly follows a robust three-tier client-server architecture with a clear separation of concerns. The presentation layer is handled by the React Native Expo app. The business logic layer consists of an Express API Gateway housing the ASA engine and controllers. The data access layer utilizes Drizzle ORM to communicate with a Neon PostgreSQL database.

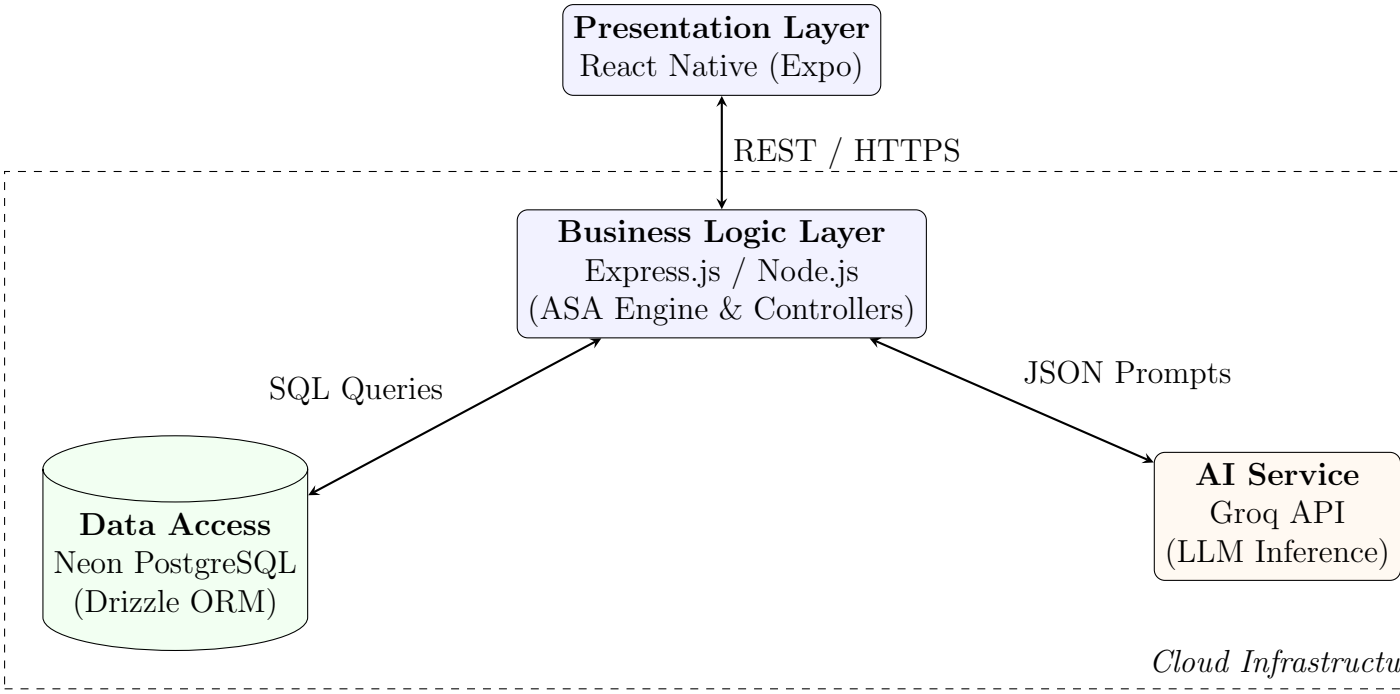


Figure 3.1: Three-Tier System Architecture of Spendly

3.1.1 Architecture Explanation

The system architecture follows a three-tier design consisting of presentation, business logic, and data layers.

The **Presentation Layer** is implemented using React Native and Expo, providing a responsive and cross-platform user interface. It handles user interactions and displays analytics dashboards and AI responses.

The **Business Logic Layer** is powered by Node.js and Express. It includes controllers for handling API requests, the ASA analytics engine for financial insights, and the AI integration module for communication with the Groq API.

The **Data Layer** uses Neon PostgreSQL, accessed via Drizzle ORM. This layer ensures data consistency, relational integrity, and optimized query performance using rollup tables.

Additionally, the system integrates external services such as:

- Groq API for AI-based financial coaching
- Expo Print for PDF report generation
- JWT for secure authentication

This modular architecture ensures scalability, maintainability, and high performance.

3.2 System Workflow

The overall workflow of Spendly can be summarized in the following steps:

1. User logs into the system using secure authentication.
2. User performs actions such as adding transactions or setting budgets.
3. Backend processes the request and updates the database.
4. ASA engine computes analytics such as discipline score and spending trends.
5. AI Coach generates personalized insights using real-time data.
6. Results are returned to the frontend and displayed to the user.

3.3 Detailed Functional Requirements

The system requirements are derived directly from the SRS document.

3.3.1 Authentication and Profile

Users can register and log in using secure JWT-based authentication. Financial profiles store income, fixed needs, and savings goals.

3.3.2 Transaction Management

Users can add, view, and categorize transactions. Each transaction includes metadata such as intent (need/want), payment mode, and debt tracking.

3.3.3 Budget Management

Budgets are defined per category. The system tracks spending and triggers alerts when 80% of the limit is reached.

3.3.4 AI Financial Coach

The AI module processes user queries, injects real-time analytics, and returns structured JSON responses to perform actions like adding transactions or setting budgets.

3.3.5 ASA Analytics Engine

The ASA engine computes:

- Discipline score (0–100)
- Spending predictions
- Behavioral insights
- Budget alerts

3.3.6 Additional Features

Recurring transactions, savings goals, habit streak tracking, and report generation are also supported.

3.4 External Interface Requirements

3.4.1 User Interface

The mobile application is built using React Native with a modern UI consisting of dashboards, charts, and chat interfaces.

3.4.2 Software Interfaces

- Neon PostgreSQL for database storage
- Groq API for AI inference
- Drizzle ORM for database operations
- Express.js for backend APIs

3.4.3 Communication Interface

All communication between frontend and backend occurs over HTTPS using REST APIs with JSON format.

3.5 API Specification

Table 3.1: Core API Route Specifications

Method	Endpoint	Auth	Description
POST	/auth/register	No	Create user with bcrypt hash; return JWT + user object
POST	/auth/login	No	Verify credentials; return JWT (30d expiry)
GET	/profile	Yes	Return user's financial profile data
POST	/profile	Yes	Upsert base_income, fixed_needs, goals
GET	/transactions	Yes	List all transactions for authenticated user
POST	/transactions	Yes	Add transaction (amount, type, intent, paymentMode)
GET	/dashboard	Yes	Returns full ASA analytics snapshot
POST	/chat	Yes	AI coach integration; returns action & text payload
POST	/goals/:id/allocate	Yes	Add progress to specific savings goal
GET	/reports/generate	Yes	Fetch structured data for native PDF formatting

3.6 Database Design

The system uses a relational database with 13 tables including:

- users
- transactions
- budgets
- goals
- recurring_configs
- habit_metrics
- monthly_summaries
- behavior_aggregates

The schema ensures data normalization, fast query performance using rollup tables, and user-specific data isolation.

3.7 System Design Models

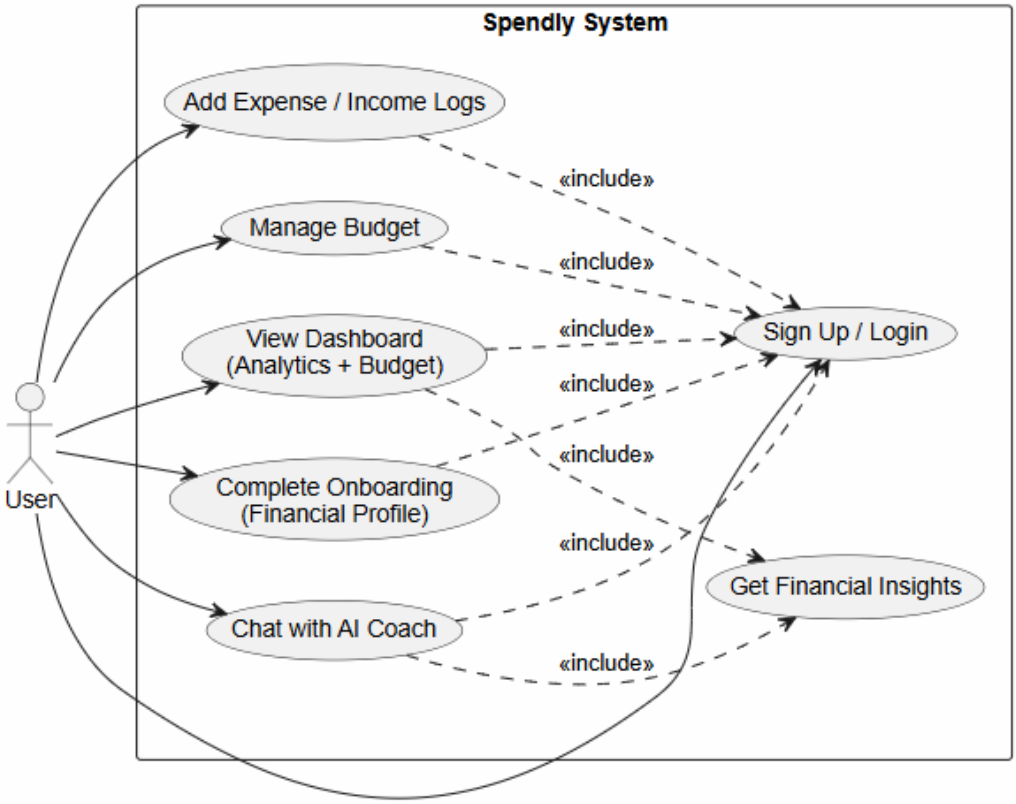


Figure 3.2: Use Case Diagram for Spendly System

3.7.1 Use Case Description

The use case diagram in Figure 3.2 represents the interaction between different actors and the Spendly system.

The primary actor is the **User**, who performs core functionalities such as managing transactions, setting budgets, tracking goals, generating reports, and interacting with the AI coach. These operations form the central workflow of the application.

The **Power User** extends the capabilities of a normal user by leveraging advanced features such as recurring transaction management and deeper analytics insights.

The **Admin/Developer** actor is responsible for system-level operations such as maintenance, deployment, and monitoring. Although not directly interacting with business features, this role ensures system reliability.

Most functionalities include authentication (`Register/Login`), indicating that all operations are secured and require user identity validation before execution.

The diagram also highlights the modular design of the system, where each feature operates independently but contributes to the overall financial tracking ecosystem.

3.8 Data Flow Overview

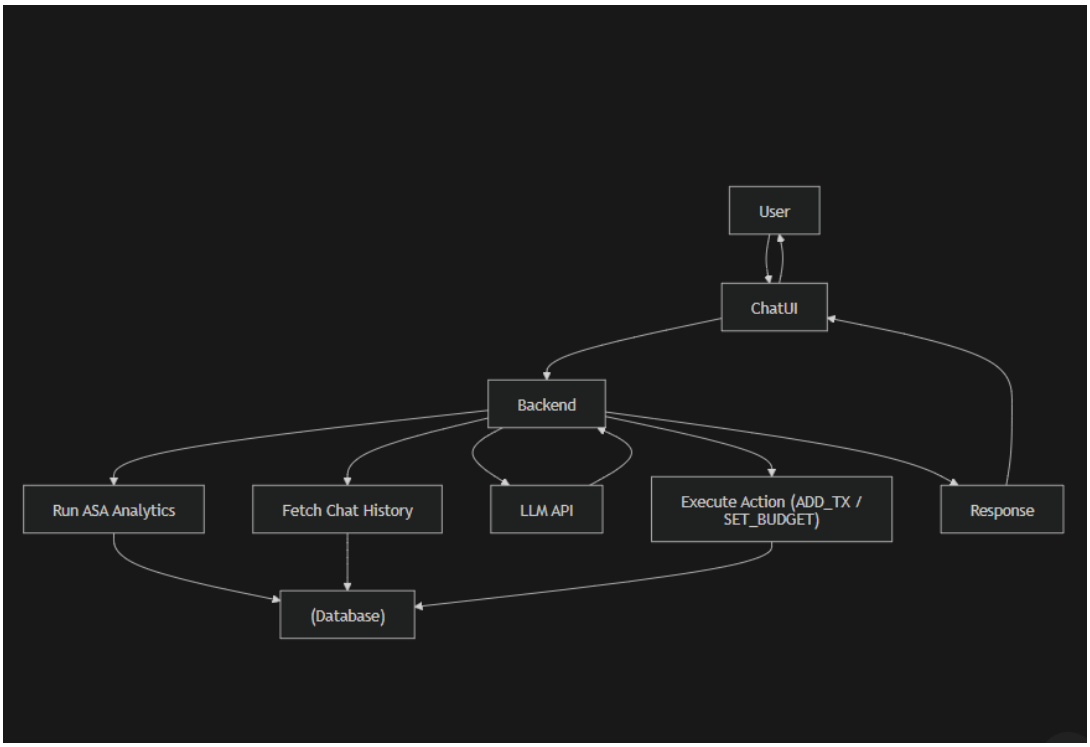


Figure 3.3: Data Flow Diagram (Level 2) of Spendly System

3.8.1 Data Flow Explanation

The data flow diagram illustrates how information moves within the Spendly system.

User inputs such as transactions, budgets, and chat messages are first captured by the frontend application. These inputs are then transmitted to the backend server through REST APIs.

The backend processes requests using business logic modules such as transaction controllers and the ASA analytics engine. For AI-based interactions, the system communicates with the Groq API, where prompts enriched with financial data are processed.

All persistent data is stored in the Neon PostgreSQL database, ensuring consistency and scalability. The processed results are then returned to the frontend in JSON format and displayed to the user.

This structured flow ensures separation of concerns, efficient processing, and real-time feedback.

3.9 Design Decisions

Several key design decisions were made during development:

- Use of React Native to maintain a single codebase for iOS and Android.
- Adoption of serverless PostgreSQL (Neon) for scalability.
- Integration of AI through Groq API for low-latency responses.

- Use of rollup tables to optimize dashboard performance.
- Strict JSON-based AI responses to enable action-driven automation.

3.10 Non-Functional Requirements

3.10.1 Performance

- API response time less than 500ms
- AI response time less than 3 seconds

3.10.2 Security

- Password hashing using bcrypt
- JWT-based authentication
- Secure API communication over HTTPS

3.10.3 Scalability

The system supports horizontal scaling due to stateless architecture and serverless database.

3.10.4 Usability

The application is designed for ease of use, allowing users to log expenses within seconds.

Chapter 4

Results and Discussion

4.1 Experimental Setup / Environment

The backend was executed on Node.js v16+ hosted on cloud platforms, connected to a Neon serverless instance. The frontend was validated on physical Android (API 26+) and iOS 13+ devices utilizing the Expo SDK runtime.

4.2 Results

The system successfully met the core objectives outlined in the SRS.

Table 4.1: System Performance Evaluation vs Target NFRs

Metric	**Target**	**Observed Result**
API Response Time (CRUD)	$\leq$ 500ms	$\sim$120ms - 250ms
AI Coach Latency (Groq)	$\leq$ 3000ms	$\sim$1800ms
Dashboard Analytics Query	$\leq$ 300ms	$\sim$80ms (via Rollup tables)
UI Frame Rate	60 fps	60 fps

4.3 Discussion

The integration of the ASA engine directly into the LLM context prompt proved highly successful. By providing the AI with data such as `wantsRemaining` and `disciplineScore`, the LLM was able to generate responses that actively prevented impulse buying. The latency of the AI coach averaged 1.8 seconds, comfortably below the 3-second non-functional requirement limit, ensuring the chat interface felt natural and conversational. The use of Drizzle ORM alongside pre-computed rollup tables successfully eliminated performance bottlenecks during the rendering of the analytics dashboard.

Chapter 5

Conclusion

5.1 Summary

This project successfully developed Spendly, a comprehensive AI-powered expense habit coach. The application resolves the limitations of traditional, passive ledger apps by implementing the Average Spending Analysis (ASA) engine and integrating the Groq LLM API. The system allows users to seamlessly log transactions, track consecutive day streaks, manage dynamic budgets, and receive proactive coaching. The three-tier architecture leveraging React Native, Express, and Neon PostgreSQL proved highly scalable and met all functional and performance requirements defined in the SRS.

5.2 Future Work

Several pathways remain open for future enhancement (v2.0). First, integrating external banking APIs (such as Plaid) would allow for the automatic import of transactions, further reducing user friction. Second, implementing push notifications for budget threshold alerts and streak reminders would improve user retention. Finally, extending the system to support multi-currency conversion and multi-device real-time syncing via WebSockets would increase the software's global applicability.

Bibliography

[1] Department of Computer Science & Engineering, *B.Tech 3rd Year Course Materials (Software Engineering Lab)*, National Institute of Technology Mizoram, 2026.

[2] Expo. (2026). *Expo Documentation*. [Online]. Available: `https://docs.expo.dev/`. [Accessed: 27 Apr. 2026].

[3] Groq Inc. (2026). *Groq API Documentation*. [Online]. Available: `https://console.groq.com/docs`. [Accessed: 27 Apr. 2026].

[4] Neon.tech. (2026). *Neon Serverless PostgreSQL Documentation*. [Online]. Available: `https://neon.tech/docs`. [Accessed: 27 Apr. 2026].

Appendix A

Technical Implementation Details

Appendix A: Adaptive Spending Algorithm (ASA Engine)

The ASA Engine is the core analytics module responsible for transforming raw transaction data into meaningful financial insights. It operates as a centralized computation layer invoked during dashboard loads and AI interactions.

The primary entry point is the `runFullAnalytics(userId)` function, which aggregates multiple analytical components into a unified response object.

Core Responsibilities

- Compute adaptive budget allocation (Needs, Wants, Savings)
- Analyze user spending behavior patterns
- Generate real-time alerts and warnings
- Predict future financial outcomes
- Provide AI-ready structured insights

Adaptive Budget Formula

The system follows a dynamic budgeting strategy:

- Needs: 50–60% of income (ensuring fixed expenses are covered)
- Savings: Maximum of target goal or 20% of income
- Wants: Remaining balance after needs and savings

This logic ensures flexibility while maintaining financial discipline.

Behavior Analysis Metrics

The ASA engine computes advanced behavioral indicators:

- **Impulse Buying Score**: Based on low-value frequent purchases
- **Emotional Spending Score**: Detected via clustered transactions
- **Late Night Spending**: Percentage of transactions after 10 PM
- **Weekend Overspending**: Ratio of weekend vs weekday expenses
- **Discipline Score**: Composite score (0–100)

These metrics enable personalized financial coaching.

Appendix B: AI Financial Coach Pipeline

The AI Financial Coach is implemented using a structured interaction pipeline that integrates real-time analytics with a Large Language Model (Groq API).

Processing Flow

1. User sends a message via chat interface
2. Backend invokes `runFullAnalytics(userId)`
3. Analytics snapshot is injected into system prompt
4. Last 12 chat messages are retrieved for context
5. Request is sent to Groq LLM with strict JSON response format
6. AI response is parsed and mapped to system actions
7. Backend executes actions such as:
    - ADD_TX (log transaction)
    - SET_BUDGET
    - UPDATE_GOAL
8. Updated analytics snapshot is returned to frontend

Structured AI Response

The AI strictly returns JSON in the following format:

```
{
  "action": "ADD_TX | SET_BUDGET | UPDATE_GOAL",
  "amount": number,
  "category": string,
  "intent": "need | want",
  "reply": "Natural language response"
}
```

This ensures deterministic system behavior and avoids ambiguity.

Key Design Decision

Unlike traditional chatbots, the AI is tightly coupled with system actions, making it an executable assistant rather than a passive advisor.

Appendix C: Alert Generation System

The system includes a rule-based alert engine that evaluates financial conditions and generates actionable notifications.

Alert Types

- Budget Threshold Alerts (80%, 90%, 100%)
- Spending Spike Alerts
- Low Savings Warnings
- Subscription Overuse Detection
- Behavioral Alerts (late-night, impulse buying)

Severity Levels

- Red: Critical issues (overspending, low savings)
- Yellow: Warnings (approaching limits)
- Green: Positive reinforcement

Alerts are prioritized and returned to the frontend for display and AI coaching integration.

Appendix D: Database Design Notes

The system uses a relational schema with 13 tables designed for performance and scalability.

Key Design Strategies

- UUID-based primary keys for global uniqueness
- Normalized schema with foreign key relationships
- Rollup tables for performance optimization

Performance Optimization

To avoid expensive queries on large datasets:

- Monthly summaries are precomputed
- Category-level aggregates are stored separately
- Behavior aggregates reduce runtime calculations

This ensures fast dashboard rendering even with large datasets.

Appendix E: Sample API Interaction Flow

Example: Adding a Transaction via AI

1. User Input: "Spent 200 on food via UPI"
2. AI Response:

```
{
  "action": "ADD_TX",
  "amount": 200,
  "category": "Food",
  "intent": "want"
}
```

3. Backend stores transaction in database
4. ASA engine recomputes analytics
5. Updated snapshot returned to UI